

- Setup **identity** Steps:

1. install packages,
2. add dB context file,
3. configure in program.cs (Initially, I added 3 roles admin, risk owner, employee)
4. add local dB connection in appsettings
5. manual add to login/register views and controller pages

- Setup **database & migration**

1. Create Data Folder , add ApplicationDbContext.cs , add inheritance :DbContext or :IdentityDbContext

*DbContext is a completely blank canvas, while IdentityDbContext is a pre-structured, ready-to-use blueprint for user management and security.

2. Configure the Constructor, Models, and Relations
3. Edit appsettings.json (Add Connection String)
4. Configure Program.cs (Register Services & Middleware)

- Setup **language**

1. **Core Setup (Program.cs):** Register localization services and add the localization middleware to automatically detect and apply the user's current language.**Translation Files (.resx):** Create a **Resources** folder to store words and their translations for each language (e.g., one file for English, another for Arabic).
2. **Organization Methods:**
 - a. **Shared Resources:** A single file handles translations for the entire project (recommended and easiest).
 - b. **Specific Resources:** Separate translation files created specifically to match each individual Controller or View.
3. **Usage in Controllers:** Inject `IStringLocalizer` to fetch the correct translated text based on the active user culture.
4. **Usage in Views:** Inject `IViewLocalizer` at the top of your Razor pages (.cshtml) to display UI text in the user's selected language.
5. **Language Switcher:** Build a simple feature that lets users manually select their language, saving their preference in a browser Cookie for future visits.

- To make ComplaintCategory checkbox

First solution

1. Edit data type in model
 2. Add this in appdbcontext to convert from list to long text "this step to can save it in DB without it you can save a list in one cell"
- ```
base.OnModelCreating(modelBuilder);
// داخل قاعدة البيانات تلقائياً JSON أخبر الإطار بتحويل القائمة إلى نص
modelBuilder.Entity<ComplaintModel>().Property(e => e.ComplaintCategory)
.HasConversion(v => JsonSerializer.Serialize(v, (JsonSerializerOptions)null), v =>
JsonSerializer.Deserialize<List<string>>(v, (JsonSerializerOptions)null) ?? new
List<string>(), new ValueComparer<List<string>>(c1, c2) => c1.SequenceEqual(c2), c
=> c.Aggregate(0, (a, v) => HashCode.Combine(a, v.GetHashCode())), c => c.ToList());
```

Second solution

1. Leave the datatype as string in model
  2. Create a separate viewmodel to save Category as list
  3. Convert it to string in controller
- Create

```
_db.ComplaintCategory = string.Join(",", viewModel.ComplaintCategory);
```
  - Details

```
viewModel.ComplaintCategory = dbModel.ComplaintCategory?.Split(',').ToList() ?? new
List<string>();
```

- Attaching file

1. In viewmodel : add `List<IFormFile> Attachments { get; set; }`
2. Edit create view + add `enctype="multipart/form-data"` to `<form>` to accept file and text
3. In model : add `string? AttachmentPath`
4. In controller :
  - Add `private readonly IWebHostEnvironment _webHostEnvironment; "to attach with wwwroot"`
5. In details view:

